Deep Compression

Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding

Group 13
EEA Winter 2025-26

The Deployment Challenge: Why Compress?

Storage Constraints

- SOTA DNNs are massive:
  - AlexNet: ~240MB
  - VGG-16: ~550MB
- Hard to deploy on mobile (App Store limits, storage space).

The "Energy Wall"

- Energy is dominated by **memory access**, not computation(0.9pJ).
- Fetching from off-chip DRAM (640pJ) is **~100x** more expensive than on-chip SRAM (5pJ).

Goal: Fit Model in SRAM without any loss in accuracy

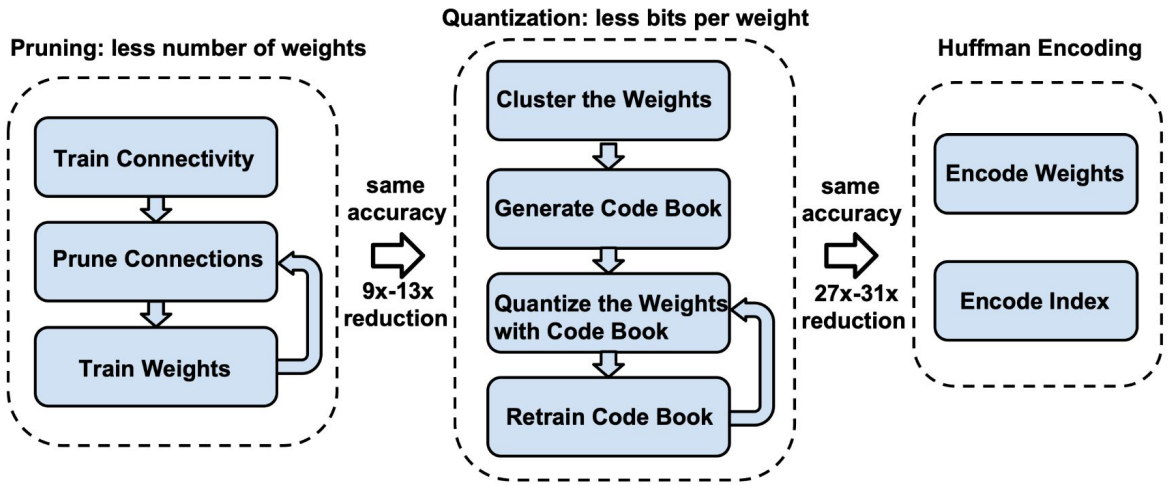
To avoid the energy penalty of DRAM, the entire model must fit into the on-chip cache.

We can then deploy on a power and storage constrained mobile device

Deep Compression: A 3-Stage Pipeline

1. **Pruning:** Remove, i.e. set to zero, redundant connections. Retain only the informative connections.
2. **Quantization:** Reduce bits required to store each weight using weight sharing: storing only the codebook and indices.
3. **Huffman Coding:** Lossless compression taking advantage of the biased distribution of final weights.

With no final loss in accuracy



Stage 1: Pruning Methodology

1. Train

Standard network training.

1	1	2
7	8	2
0	6	0

Status: High Accuracy, High Connections.

2. Prune

Remove connections with weights below a threshold.

0	0	0
7	8	0
0	6	0

Status: Low Accuracy, Low Connections.

3. Retrain

Fine-tune remaining sparse weights to recover accuracy.

0	0	0
8	9	0
0	5	0

Status: High Accuracy, Low Connections.

Efficient Storage: CSR/CSC & Relative Indexing

Storing Sparse Matrices

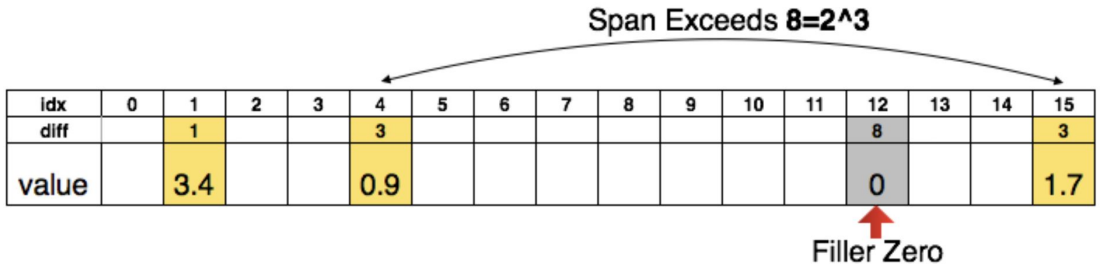
Use Compressed Sparse Row (CSR) or Column (CSC) format to store the sparse structure obtained after pruning.

With CSR, need to store **2a + n + 1** numbers
as opposed to
~n x n numbers

a is the no. of non-zero elements, **n** is no. of rows

Relative Indexing

- Instead of absolute positions, store the **relative index difference**.
- **CONV Layers:** 8 bits for index diff.
- **FC Layers:** 5 bits for index diff.
- If difference > bound, add a zero filler.



Stage 2: Trained Quantization

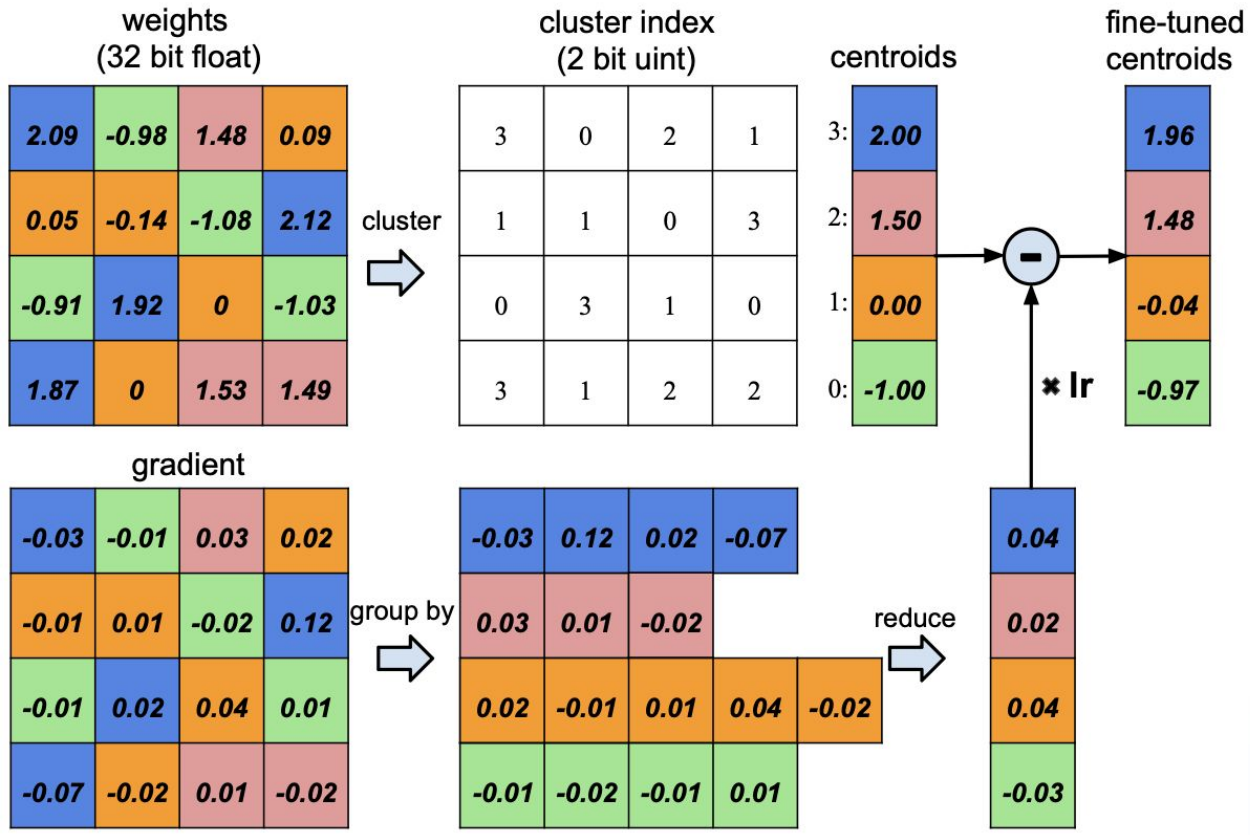
Weight Sharing

We restrict the weights to a small set of shared values (centroids) determined by **k-means clustering**.

Fine-tuning Centroids

- Gradients are grouped by cluster.
- We sum the gradients in each cluster to update the single shared centroid.
- This allows learning the optimal quantization values.

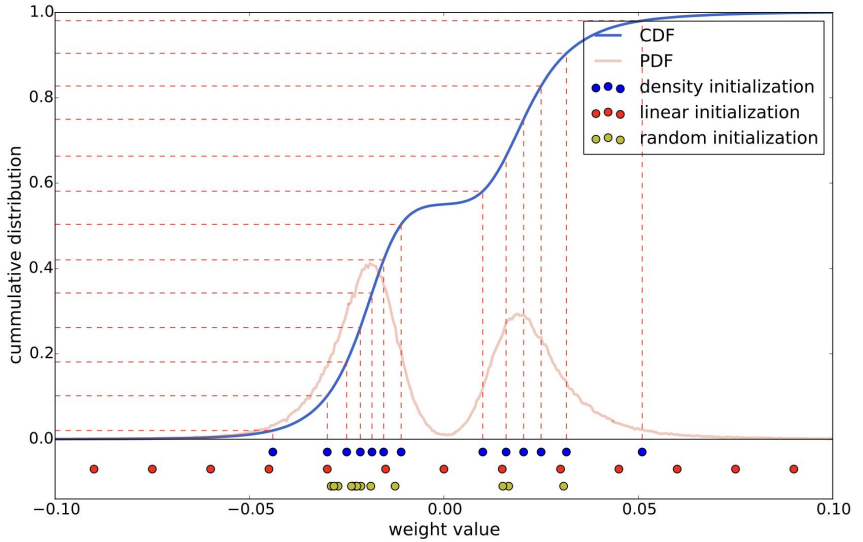
Ex: r = (16*32)/(16*2 + 4*32) = 3.2



Quantization Details

How many clusters?

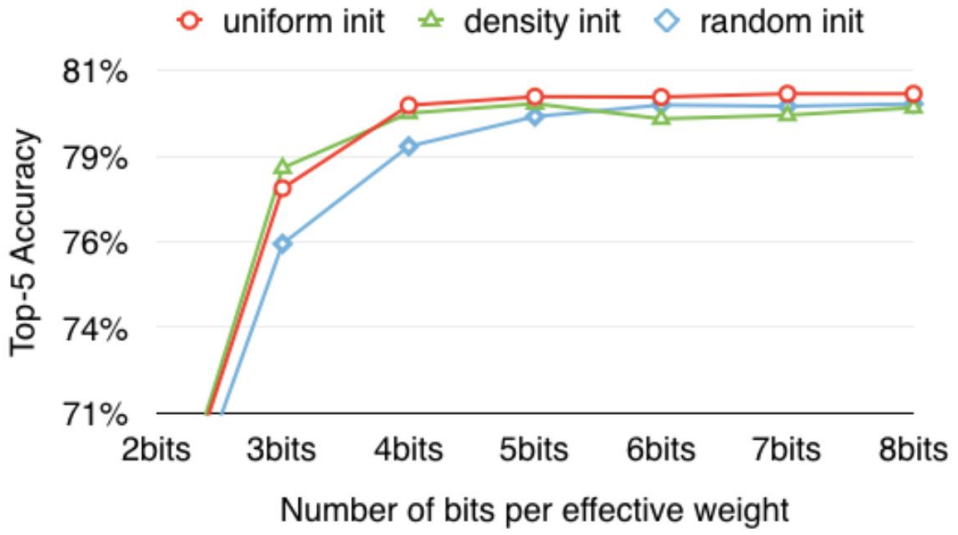
- **CONV Layers:** 8 bits (256 shared weights). Feature extraction is sensitive.
- **FC Layers:** 5 bits (32 shared weights). Feature classification is less sensitive.



Centroid Initialization

How initial centroids are picked matters:

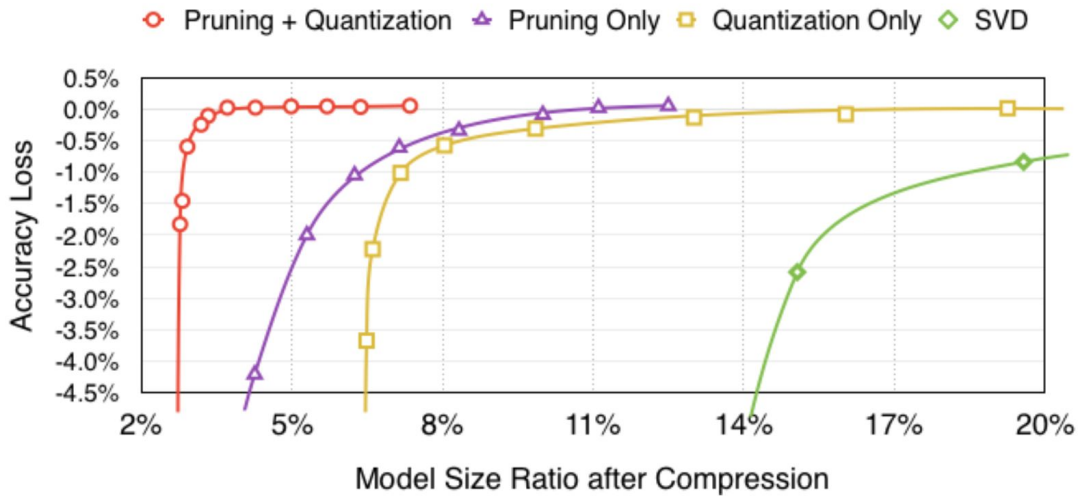
- **Linear Initialization:** Works best.
- Why? It preserves large weights, which are critical, unlike density-based initialization which clusters around the peaks (small values).



Synergy: Pruning + Quantization

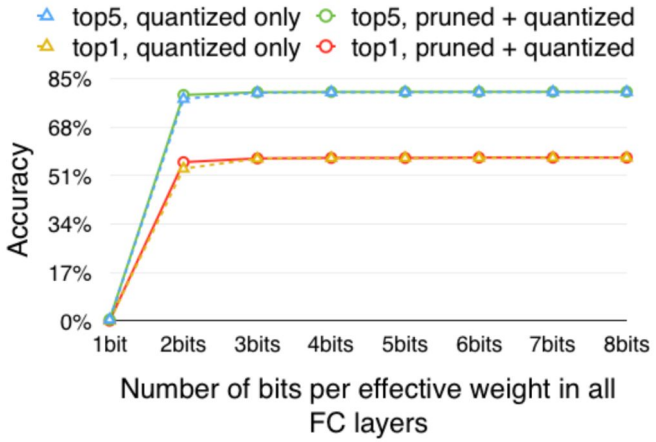
Better Together

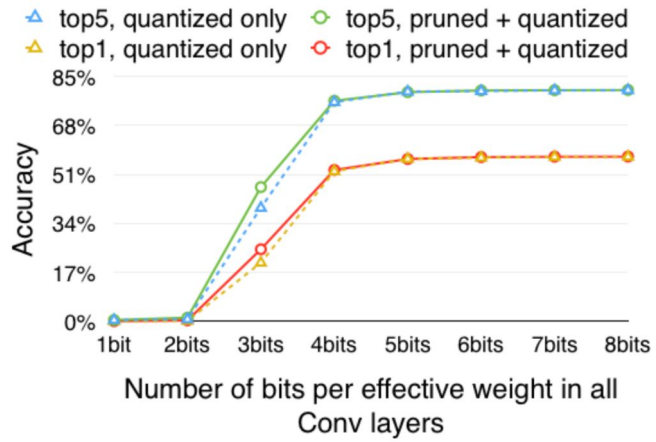
As shown in the graph (red line), combined compression achieves **3%** of original size with no accuracy drop, far outperforming individual methods.



Pruning does not hurt Quantization

Accuracy begins to drop at the same number of quantization bits whether or not the network has been pruned. Although pruning made the number of parameters less, quantization still works well, or even better than in the unpruned network.





Stage 3: Huffman Coding

Lossless Compression

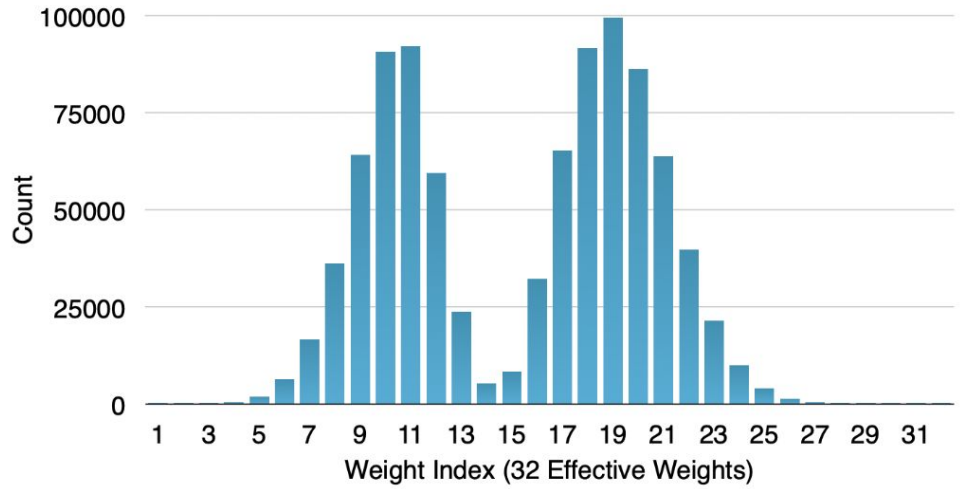
Applied after pruning and quantization. Uses variable length coding to encode symbols

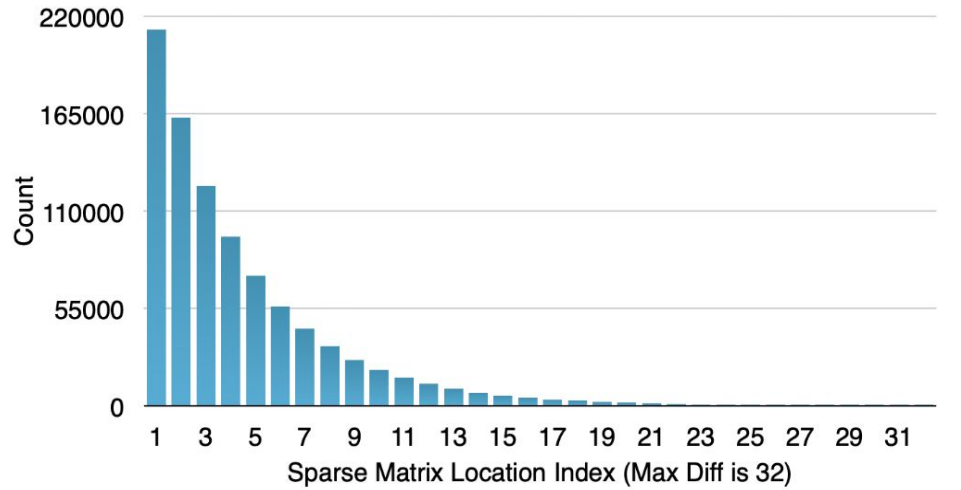
Key Observation

- Weights and sparse indices are **non-uniformly distributed**.
- Many weights cluster around peaks and relative indices in the sparse matrix are usually small.

Result

Variable-length coding saves an additional **20-30%** storage.





Experimental Results: Storage & Accuracy

Deep Compression achieves massive reduction with **Zero Accuracy Loss**.

Network	Original Size	Compressed Size	Reduction	Accuracy (Top-1)
AlexNet	240 MB	6.9 MB	35x	-0.33%
VGG-16	552 MB	11.3 MB	49x	No Loss
LeNet-5	1.7 MB	0.04 MB	39x	-0.06 %

The model fits easily into on-chip SRAM (usually few MBs).

Layer-wise Sensitivity Analysis

Convolutional (CONV) Layers

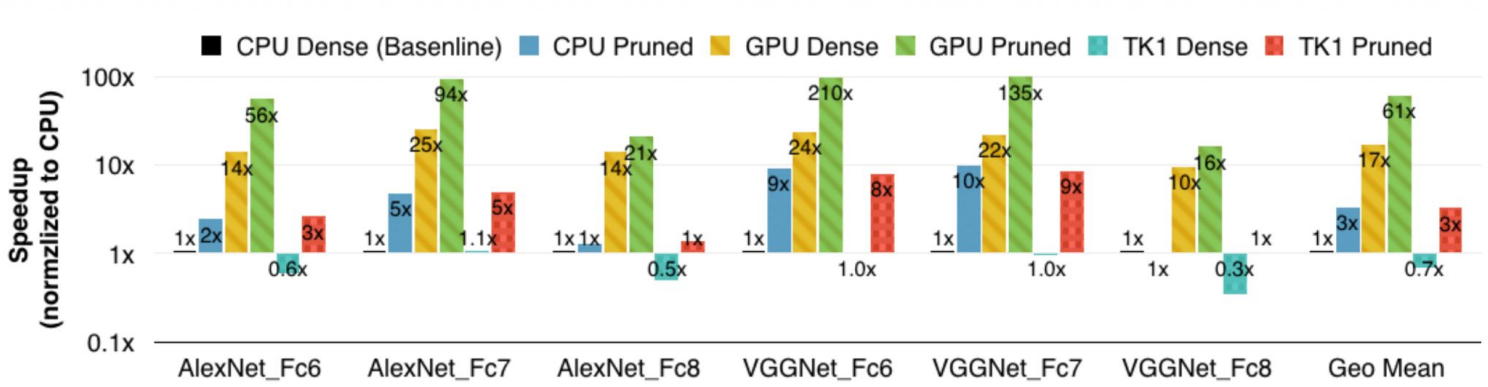
- Sensitive to precision
- Must be quantized to 8 bits
- Pruning rate: ~60-70% remains

Fully Connected (FC) Layers

- Highly redundant
- Can be quantized to 5 bits
- Pruning rate: <10% remains (massive reduction)

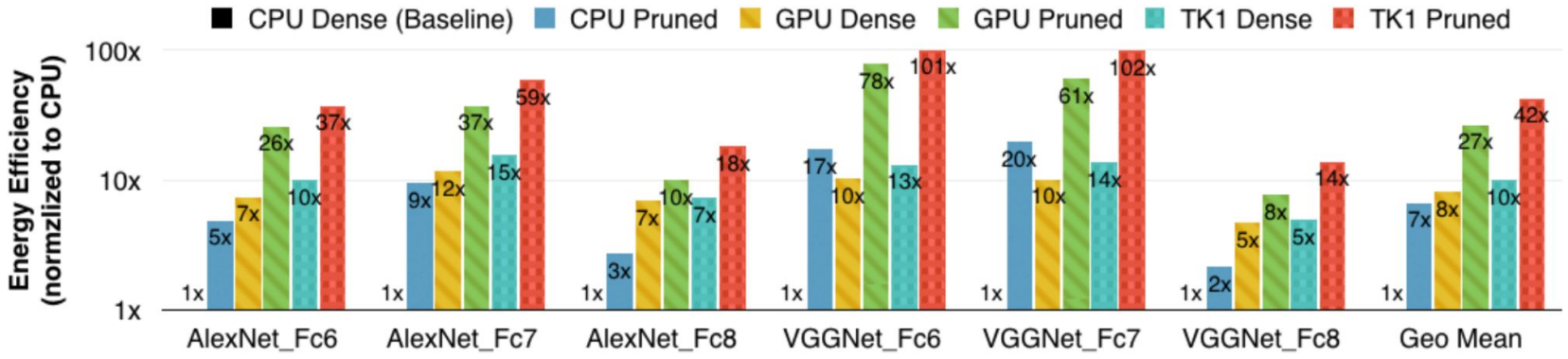
Not all layers are equal. FC layers are highly redundant and can be compressed massively (down to 5 bits, removing 90% of weights). CONV layers are more sensitive and need 8 bits

Speedup & Energy Efficiency



Speedup

3x to 4x faster on CPU, GPU, and Mobile GPU.



Energy

3x to 7x more energy efficient.

Limitations & Conclusion

Only the Pruned Network has been benchmarked

- Current standard libraries do not support.
  - Indirect matrix entry look-up
  - Relative indexing
  - Custom bit widths (e.g., 5-bit)

  So, the quantized model could not be benchmarked

Conclusion

- ✓ **Key Achievement:** Fits models into on-chip SRAM enabling energy efficient DNN's to run on mobile
- ✓ **Impact:** Reduces size by ~40x, improves energy by ~5x, with **no accuracy loss**.
- ✓ **Future:** Facilitates the use of more complex neural networks in mobile applications.

Contributors

1.Tharun D - 231097

2.Shreyansh Singh - 230989

3.Alok Kumar -240093

4.Snehasish Haldar - 231013

5. Ahmed Adnan- 230089